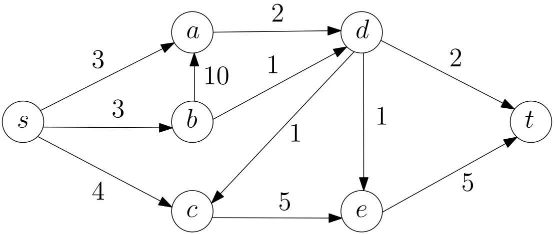
Network Flow

- Maximum Flow: Problem Formulation
- Maximum Flow: Upper Bound
- Maximum Flow: Adding flow along paths
- Residual Network and Augmenting Path
- Ford-Fulkerson Algorithm – Max-Flow-Min-Cut Theorem
- Edmond-Karp Algorithm
- Maximum Flow: Variants and Applications

Imdad ullah Khan

Network Flows : Motivation

A network of pipelines along which oil can be sent



We want to ship as much oil from s to t as possible

There are two restrictions

- A pipeline cannot carry more oil than weight of the corresponding edge
- No node can store any oil

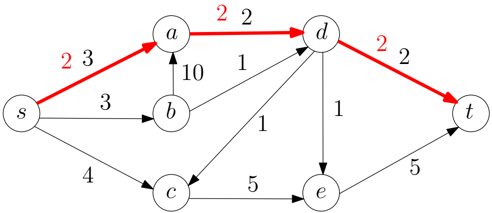
Network Flows : Motivation

A network of pipelines along which oil can be sent

We want to ship as much oil from s to t as possible

There are two restrictions

- A pipeline cannot carry more oil than weight of the corresponding edge
- No node can store any oil



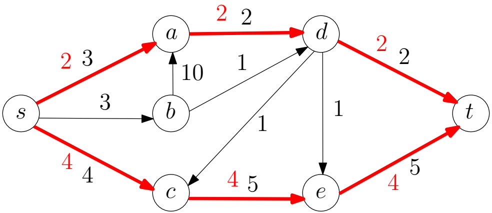
Send 2 units of flows along the path s, a, d, t

A network of pipelines along which oil can be sent

We want to ship as much oil from s to t as possible

There are two restrictions

- A pipeline cannot carry more oil than weight of the corresponding edge
- No node can store any oil



Send 2 units of flows along the path s, a, d, t

Another 4 units of flow can go through the path s, c, e, t

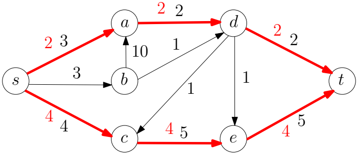
Network Flows : Motivation

A network of pipelines along which oil can be sent

We want to ship as much oil from s to t as possible

There are two restrictions

- A pipeline cannot carry more oil than weight of the corresponding edge
- No node can store any oil

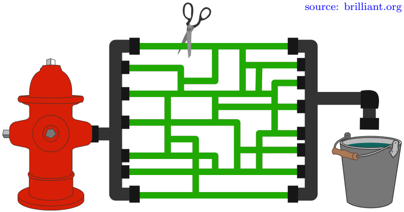


- Is this the best flow (the largest amount of oil that can be shipped)?
- How do we measure the size of flow?
- How do we determine that a given flow is the maximum possible?

Max Flow : Problem Formulation

A flow network: A directed graph with weights on edges

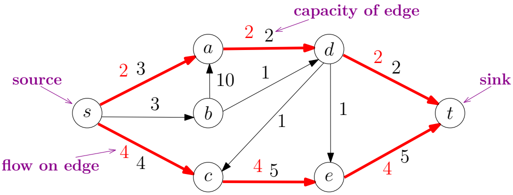
- Models a transportation network
- Edges can carry traffic and nodes switch traffic between edges
  - highway network (vertices: intersections/interchanges, edges: roads)
  - communication network (vertices: switches/routers, edges: links)
  - fluid networks (vertices: junctures where pipes are plugged, edges: pipelines)
- Edge weights are capacities of links (max traffic they can carry)



source: brilliant.org

Max Flow : Problem Formulation

A flow network: A directed graph with weights on edges



- capacity of the edge uv: $e = uv$ is associated with $c_e = c_{uv} \in \mathbb{R}^+$
- source s: $deg^-(s) = 0$ is the traffic generator
- sink t: $deg^+(t) = 0$ is the traffic consumer
- A $s - t$ flow: assignment to each edge e a flow $f_e \in \mathbb{R}^+$ with $f_e \leq c_e$
- flow $f : E \to \mathbb{R}^+$ satisfying the capacity and storage constraints

Max Flow : Problem Formulation

Given a flow network $G = (V, E, c)$, $c : E \to \mathbb{R}^+$

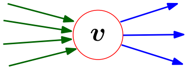
$s \in V$ a source and $t \in V$ a sink

$f : E \to \mathbb{R}^+$ $\quad (f_e = f(e))$ $\quad$ is a flow if it satisfies

1. (capacity constraints):

$$u \xrightarrow{f_e < c_e} v$$

$$\forall\, e \in E : \quad 0 \leq f_e \leq c_e$$



2. (flow conservation constraints):

$$\forall\, v \in V, v \neq s, t : \quad \underbrace{\sum_{e \text{ into } v} f_e}_{\substack{\text{total flow} \\ \text{incoming to } v}} = \underbrace{\sum_{e \text{ out of } v} f_e}_{\substack{\text{total flow} \\ \text{outgoing from } v}}$$

Max Flow : Problem Formulation

Given a flow network $G = (V, E, c)$, $c : E \to \mathbb{R}^+$

$s \in V$ a source and $t \in V$ a sink

$f : E \to \mathbb{R}^+$ $\quad (f_e = f(e))$ $\quad$ is a flow

For $v \in V$, $\quad f^{out}(v) := \sum_{\substack{e \text{ outgoing} \\ \text{from } v}} f_e$ $\quad$ and $\quad f^{in}(v) := \sum_{\substack{e \text{ incoming} \\ \text{to } v}} f_e$

For $X \subset V$, $\quad f^{out}(X) := \sum_{\substack{e \text{ outgoing} \\ \text{from } X}} f_e$ $\quad$ and $\quad f^{in}(X) := \sum_{\substack{e \text{ incoming} \\ \text{to } X}} f_e$

Max Flow : Problem Formulation

Given a flow network $G = (V, E, c)$, $c : E \to \mathbb{R}^+$

$s \in V$ a source and $t \in V$ a sink

$f : E \to \mathbb{R}^+$ $\quad (f_e = f(e))$ $\quad$ is a flow if it satisfies

1. capacity constraints $\quad \forall\, e \in E \;:\; 0 \leq f_e \leq c_e$
2. flow conservation constraints $\quad \forall\, v \in V, v \neq s, t \quad f^{out}(v) = f^{in}(v)$

The size of flow f is total flow sent out from s

$$size(f) = \sum_{\substack{e \text{ outgoing} \\ \text{from } s}} f_e = f^{out}(s)$$

$$size(f) = f^{out}(s) = f^{in}(t)$$

Max Flow : Problem Formulation

Input: A flow network $G = (V, E, c)$, $c : E \to \mathbb{R}^+$

$s \in V$ a source and $t \in V$ a sink

$f : E \to \mathbb{R}^+$ ($f_e = f(e)$) is a flow if it satisfies

1. capacity constraints $\forall\, e \in E \;:\; 0 \leq f_e \leq c_e$
2. flow conservation constraints $\forall\, v \in V, v \neq s, t \quad f^{out}(v) = f^{in}(v)$

$$size(f) = f^{out}(s) = f^{in}(t)$$

Output: A flow f of maximum size

Network Flow

- Maximum Flow: Problem Formulation
- Maximum Flow: Upper Bound
- Maximum Flow: Adding flow along paths
- Residual Network and Augmenting Path
- Ford-Fulkerson Algorithm – Max-Flow-Min-Cut Theorem
- Edmond-Karp Algorithm
- Maximum Flow: Variants and Applications

Imdad ullah Khan

Max Flow: Problem Formulation

Input: A flow network $G = (V, E, c)$, $c : E \rightarrow \mathbb{R}^+$

$s \in V$ a source and $t \in V$ a sink

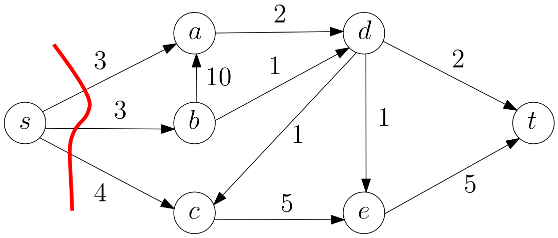
$f : E \rightarrow \mathbb{R}^+$ $(f_e = f(e))$ is a flow if it satisfies

1. capacity constraints $\quad \forall\, e \in E \;:\; 0 \leq f_e \leq c_e$
2. flow conservation constraints $\quad \forall\, v \in V, v \neq s, t \quad f^{out}(v) = f^{in}(v)$

$$size(f) = f^{out}(s) = f^{in}(t)$$

Output: A flow f of maximum size

Max Flow: Upper Bound

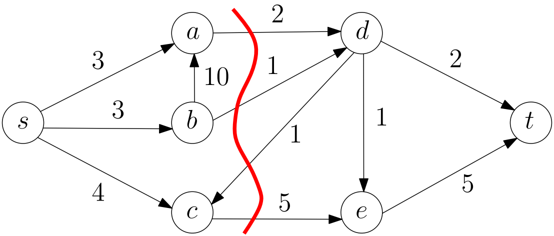


Consider the cut $\left[\{s\}, \overline{\{s\}}\right]$

Any flow generated from s has to go through one of the cut edges

Hence, no flow can be of size bigger than $3 + 3 + 4 = 10$

Max Flow: Upper Bound



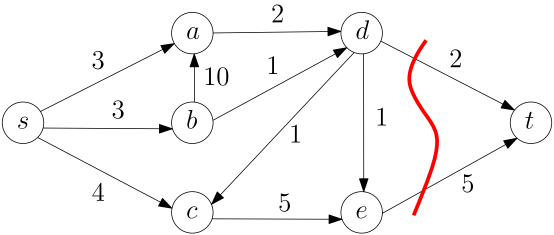
The same is true for any cut, Consider the cut $\left[\{s, a, b, c\}, \{d, e, t\}\right]$

Any flow generated from s has to go through one of the cut edges

Hence, no flow can be of size bigger than $2 + 1 + 5 = 8$

This is a tighter bound than the one we got from $\left[\{s\}, \overline{\{s\}}\right]$

Max Flow: Upper Bound



The same is true for nay cut, Consider the cut $\left[\overline{\{t\}}, \{t\}\right]$

Any flow generated from s has to go through one of the cut edges

Hence, no flow can be of size bigger than $2 + 5 = 7$

This is a tighter bound than the one we got from $\left[\{s, a, b, c\}, \{d, e, t\}\right]$

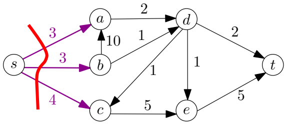
Max Flow: Upper Bound

$s-t$ cut

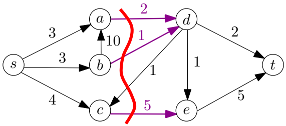
$A \subset V$, an $s-t$ cut, $\left[A, \overline{A}\right]$, is a cut in G with $s \in A$ and $t \in \overline{A}$

Capacity of an $s-t$ cut: sum of capacities of edges going from A to $\overline{A}$

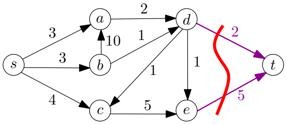
$$c\left(\left[A, \overline{A}\right]\right) = \sum_{e \text{ outgoing from } A} c_e$$



An $s-t$ cut of capacity 10



An $s-t$ cut of capacity 8



An $s-t$ cut of capacity 7

Max Flow: Upper Bound

Let f be a flow in G and let $[A, \overline{A}]$ be any $s-t$ cut in G, then

$$size(f) \leq c([A, \overline{A}])$$

Proof: Let $[A, \overline{A}]$ be any $s-t$ cut. By definition we know that

$$size(f) = f^{out}(s) = f^{out}(s) + \sum_{s \neq v \in A} \left(f^{out}(v) - f^{in}(v)\right) \qquad \text{just adding 0's}$$

$$= f^{out}(s) + \sum_{s \neq v \in A} f^{out}(v) - \sum_{s \neq v \in A} f^{in}(v)$$

$$= f^{out}(A) - f^{in}(A)$$

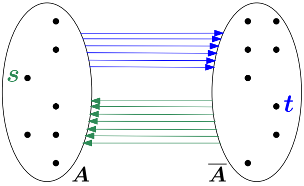
$$= \sum_{e \text{ outgoing from } A} f_e - \sum_{e \text{ incoming to } A} f_e \qquad \text{flows on other edges cancel}$$

$$\leq \sum_{e \text{ outgoing from } A} c_e - \sum_{e \text{ incoming to } A} f_e \leq \sum_{e \text{ outgoing from } A} c_e = c([A, \overline{A}])$$

Max Flow: Upper Bound

Let f be a flow in G and let $[A, \overline{A}]$ be any $s-t$ cut in G, then

$$size(f) \leq c([A, \overline{A}])$$



Tightest upper bound will come from a $s-t$ cut of minimum capacity

$[A^*, \overline{A^*}]$ be an $s-t$ cut with minimum capacity ▷ min-$s-t$-cut

We get the corollary

$$size(f) \leq c([A^*, \overline{A^*}])$$

Network Flow

- Maximum Flow: Problem Formulation
- Maximum Flow: Upper Bound
- Maximum Flow: Adding flow along paths
- Residual Network and Augmenting Path
- Ford-Fulkerson Algorithm – Max-Flow-Min-Cut Theorem
- Edmond-Karp Algorithm
- Maximum Flow: Variants and Applications

Imdad ullah Khan

Max Flow: Problem Formulation

Input: A flow network $G = (V, E, c)$, $c : E \to \mathbb{R}^+$

$s \in V$ a source and $t \in V$ a sink

$f : E \to \mathbb{R}^+$ ($f_e = f(e)$) is a flow if it satisfies

1. capacity constraints $\forall\, e \in E \;:\; 0 \leq f_e \leq c_e$
2. flow conservation constraints $\forall\, v \in V, v \neq s, t \quad f^{out}(v) = f^{in}(v)$

$$size(f) \;=\; f^{out}(s) \;=\; f^{in}(t)$$

Output: A flow f of maximum size

Let f be a flow in G and let $[A, \overline{A}]$ be any $s - t$ cut in G, then

$$size(f) \;\leq\; c([A, \overline{A}])$$

Max Flow: Algorithm

There is no known divide-and-conquer or dynamic programming algorithm for max-flow

We try a greedy strategy – build up flow little bit at a time

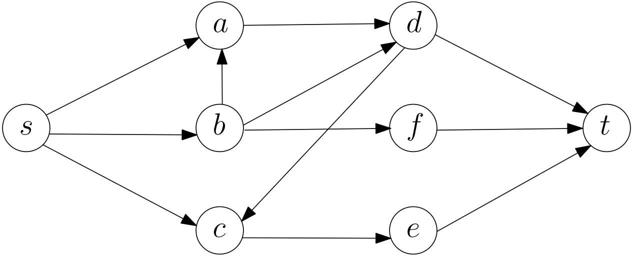
1. Start with a 0 flow

   ▷ Note: the flow f with $f_e = 0$ for all $e \in E$ satisfies both constraints

2. Add more flow to f via a $s - t$ path

Max Flow: Algorithm

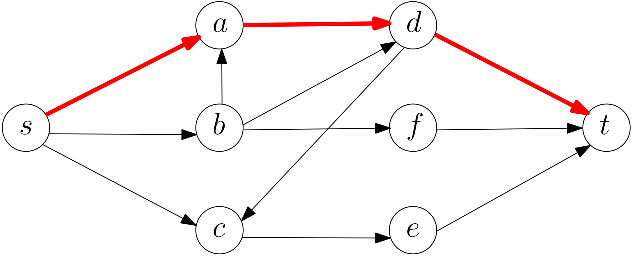
Consider the flow network – all edge capacities are 1



Max Flow: Algorithm

Consider the flow network – all edge capacities are 1

Add flow of size 1 via the $s-t$ path s, a, d, t

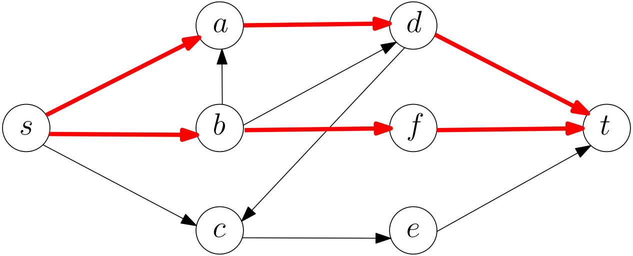


Max Flow: Algorithm

Consider the flow network – all edge capacities are 1

Add flow of size 1 via the $s - t$ path s, a, d, t

Add flow of size 1 via the $s - t$ path s, b, f, t



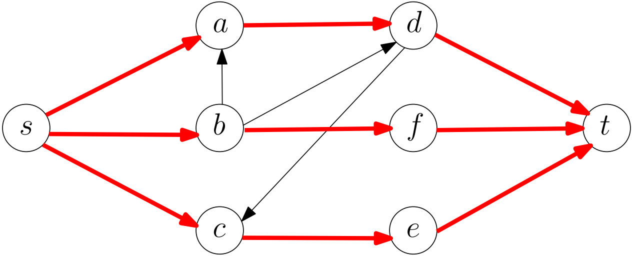
Max Flow: Algorithm

Consider the flow network – all edge capacities are 1

Add flow of size 1 via the $s - t$ path s, a, d, t

Add flow of size 1 via the $s - t$ path s, b, f, t

Add flow of size 1 via the $s - t$ path s, c, e, t



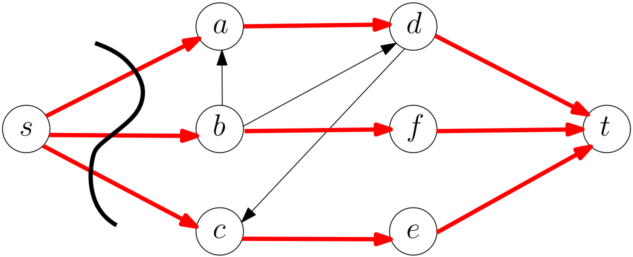
Max Flow: Algorithm

Consider the flow network – all edge capacities are 1

Add flow of size 1 via the $s - t$ path s, a, d, t

Add flow of size 1 via the $s - t$ path s, b, f, t

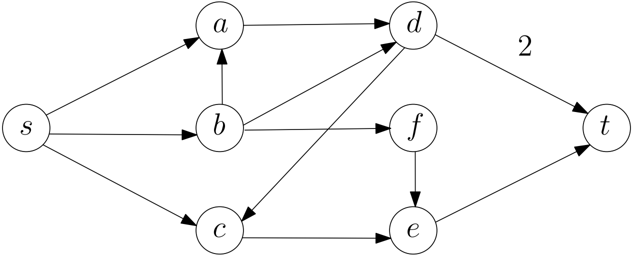
Add flow of size 1 via the $s - t$ path s, c, e, t



There is an $s - t$ cut of capacity 3, hence this flow is maximum possible

Max Flow: Algorithm

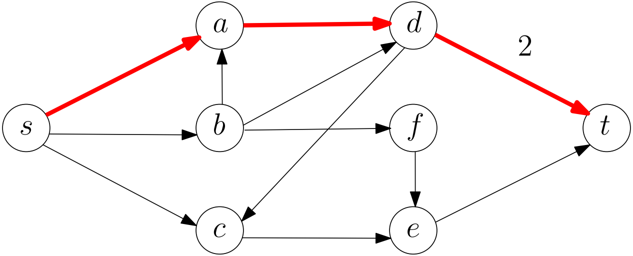
Consider the flow network – with not all capacities $= 1$



Max Flow: Algorithm

Consider the flow network – with not all capacities $= 1$

Add flow of size $1 =$ path-bottleneck via the $s - t$ path s, a, d, t

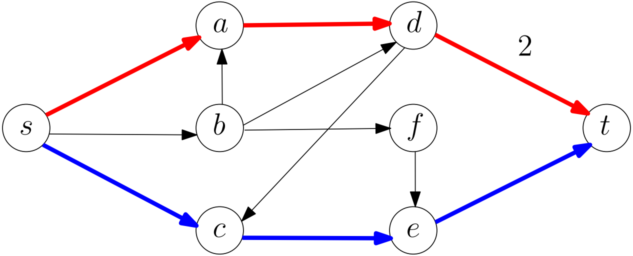


Max Flow: Algorithm

Consider the flow network – with not all capacities $= 1$

Add flow of size 1 = path-bottleneck via the $s-t$ path s, a, d, t

Add flow of size 1 via the $s-t$ path s, b, f, t



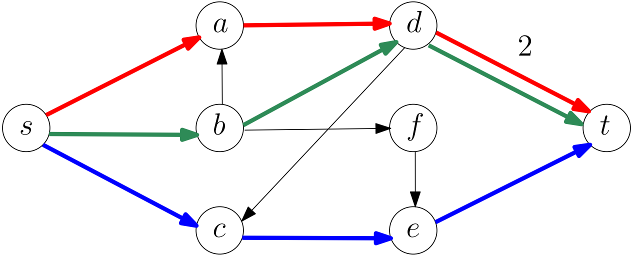
Max Flow: Algorithm

Consider the flow network – with not all capacities $= 1$

Add flow of size 1 $=$ path-bottleneck via the $s - t$ path s, a, d, t

Add flow of size 1 via the $s - t$ path s, b, f, t

Add flow of size 1 via the $s - t$ path s, c, e, t



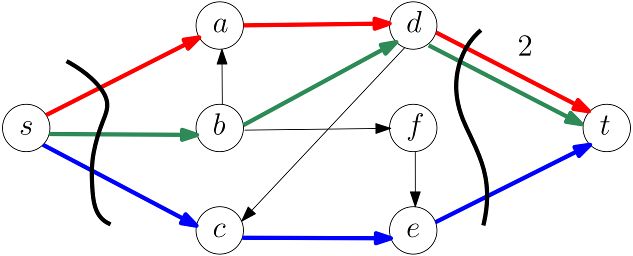
Max Flow: Algorithm

Consider the flow network – with not all capacities = 1

Add flow of size 1 = path-bottleneck via the $s-t$ path s, a, d, t

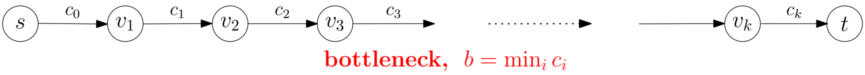
Add flow of size 1 via the $s-t$ path s, b, f, t

Add flow of size 1 via the $s-t$ path s, c, e, t

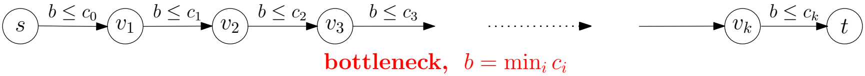


There is an $s-t$ cut of capacity 3, hence this flow is maximum possible

Max Flow: Adding Flow along a path



Adding flow along a path ensures that flow conservation constraints remain satisfied



Adding a flow along a path equal to the bottleneck of the path doesn't violate the capacity constraints

These two facts ensure that in any intermediate step of such a greedy algorithm the flow indeed is a valid flow